Peregrine: A From-Scratch Deep Learning Framework in Rust with NEON SIMD, Metal GPU, and Reverse-Mode Autograd

Evan Tam
Independent Researcher
etam103@github.com

March 2026

Abstract

We present Peregrine, a deep learning framework implemented entirely in Rust without depending on PyTorch, ONNX, or other existing ML frameworks. Peregrine provides N-dimensional tensors with reverse-mode automatic differentiation supporting approximately 220 operations, neural network layers, eight optimizers, and both CPU and GPU backends targeting Apple Silicon. The CPU backend uses Apple Accelerate BLAS and hand-tuned ARM NEON SIMD intrinsics; the optional Metal GPU backend provides 108 compute shaders with fused operator pipelines and heterogeneous CPU–GPU scheduling. We benchmark 141 operations against PyTorch 2.10, MLX 0.30, TensorFlow 2.20, JAX 0.9, and tinygrad on an Apple M1 Max, finding that Peregrine wins 128 of 141 comparisons with a geometric mean speedup of 2.3× over PyTorch. On end-to-end inference of the MUSt3R 3D reconstruction model (423M parameters), Peregrine achieves 12–36% lower latency than PyTorch. The entire framework is approximately 37,900 lines of readable Rust with 633 tests, including 23 numerical parity tests against PyTorch that pass within 10^{-4} to 10^{-7} absolute error. Peregrine is open-source under the MIT license.

1 Introduction

Modern deep learning frameworks such as PyTorch [1], TensorFlow [2], and JAX [3] have enabled rapid progress in machine learning research and deployment. However, these frameworks are large, complex software systems—PyTorch alone comprises millions of lines of C++ and Python—that depend on substantial runtime infrastructure including Python interpreters, CUDA toolchains, and framework-specific graph compilers. This complexity creates barriers to understanding, auditing, and customizing the inference pipeline, and can introduce overhead for latency-sensitive applications.

We present **Peregrine**, a deep learning framework built from first principles in Rust. Peregrine implements the full stack—tensors, automatic differentiation, neural network layers, optimizers, model serialization, and hardware-specific kernels—without depending on any existing ML framework. The design is motivated by three goals:

1. **Transparency.** Every operation is implemented in readable Rust source code. There are no opaque C++ backends, no generated code, and no proc-macro magic. A developer can trace any computation from the high-level API down to the SIMD instruction.

2. **Performance on Apple Silicon.** Rather than targeting NVIDIA GPUs via CUDA, Peregrine is optimized for Apple's ARM-based processors using Apple Accelerate (BLAS, vDSP, vForce), hand-tuned NEON SIMD intrinsics, and Metal compute shaders.

3. **Minimal dependencies.** The framework depends only on `rayon` (data parallelism), `libc` (C FFI), and optionally `objc2` (Metal bindings). This yields fast compile times, small binaries, and straightforward deployment.

Despite its small codebase (approximately 37,900 lines), Peregrine supports a broad set of operations, achieves competitive or superior performance to established frameworks on Apple Silicon, and runs real models including Grok-1 (314B parameters), DeepSeek-V3/R1 (671B parameters), MUSt3R (423M parameters), Llama 3.2, and RT-DETR.

2 Related Work

General-purpose frameworks. PyTorch [1] and TensorFlow [2] are the dominant deep learning frameworks, supporting both training and inference across multiple hardware backends. JAX [3] provides functional transformations over NumPy-like arrays compiled via XLA. All three are Python-first with C++/CUDA backends.

Apple Silicon frameworks. MLX [4] is Apple's ML framework for Apple Silicon, implemented in C++ with a Python API and unified CPU–GPU memory. Core ML provides on-device inference via the Apple Neural Engine (ANE), CPU, and GPU, but requires model conversion to the MIL intermediate representation and exposes only a deployment API.

Lightweight and Rust frameworks. tinygrad [5] is a minimalist Python framework emphasizing simplicity. In the Rust ecosystem, `candle` [6] (Hugging Face) provides tensor operations and model inference, and `burn` [7] provides a training-oriented framework with multiple backends. Peregrine differs from these in implementing all operations from scratch including SIMD kernels, the autograd engine, Metal shaders, and format parsers, with no shared backend code.

Inference engines. llama.cpp [8] provides quantized LLM inference in C++, and ONNX Runtime [9] provides cross-platform inference via the ONNX format. These are inference-only systems without training support.

3 Architecture

3.1 Tensor Representation

Peregrine tensors are N-dimensional arrays of `f32` values stored in row-major (C-contiguous) layout. Each tensor is reference-counted via `Rc<RefCell<TensorInner»`, where `TensorInner` contains:

- `data`: A contiguous `Vec<f32>` buffer.
- `shape`: The tensor dimensions as `Vec<usize>`.
- `requires_grad`: Whether this tensor participates in autograd.
- `grad`: Optional accumulated gradient (`Option<Vec<f32»`).
- `op`: The operation that produced this tensor (`Option<Op>`).

Broadcasting follows NumPy semantics: dimensions are right-aligned and expanded where one operand has size 1. All arithmetic, math, and comparison operations support broadcasting.

3.2 Automatic Differentiation

Peregrine implements reverse-mode automatic differentiation (backpropagation). Each operation records itself as an `Op` variant in the output tensor, forming a directed acyclic graph (DAG). Calling `backward()` on a scalar tensor traverses the DAG in reverse topological order, accumulating gradients via the chain rule.

The `Op` enum has approximately 220 variants, each storing the input tensors and any cached intermediate values needed for the backward pass. Gradients are accumulated (summed) when a tensor is used in multiple downstream operations.

Functional autograd utilities include `grad(f, x)` for computing $\nabla_x f(x)$, `value_and_grad(f, x)` for simultaneous forward and backward passes, and `checkpoint(f, x)` for activation checkpointing that recomputes the forward pass during backpropagation to reduce memory usage.

3.3 CPU Backend

The CPU backend combines three levels of optimization:

Apple Accelerate BLAS. Matrix multiplication uses `cblas_sgemm` from Apple's Accelerate framework, which is highly optimized for the Apple AMX coprocessor. We use row-major layout with transpose flags to avoid explicit transposition. Batched attention uses strided `cblas_sgemm` calls with pointer offsets to avoid reshaping.

Apple Accelerate vForce. Transcendental functions (exp, log, tanh, sin, cos, etc.) use Apple's vectorized vForce functions (`vvexpf`, `vvtanhf`, `vvlog1pf`, etc.), which are optimized for ARM NEON.

Hand-tuned NEON SIMD. For element-wise operations where the overhead of function calls dominates, we provide 24 hand-tuned ARM NEON kernels using Rust's `core::arch::aarch64` intrinsics. Each kernel processes 8 elements per iteration (two 128-bit NEON registers), with a scalar tail for non-aligned lengths:

```
unsafe fn vec_add_f32(a: *const f32, b: *const f32,
                      out: *mut f32, len: usize) {
    let chunks8 = len / 8;
    for i in 0..chunks8 {
        let off = i * 8;
        let va0 = vld1q_f32(a.add(off));
        let va1 = vld1q_f32(a.add(off + 4));
        let vb0 = vld1q_f32(b.add(off));
        let vb1 = vld1q_f32(b.add(off + 4));
        vst1q_f32(out.add(off), vaddq_f32(va0, vb0));
        vst1q_f32(out.add(off+4), vaddq_f32(va1, vb1));
    }
    // scalar tail for len % 8 != 0
}
```

NEON kernels cover element-wise arithmetic (add, sub, mul, div, neg, abs, scale, relu), reduction operations (sum, mean, max, min, var, logsumexp, cumsum, argmax), and activation functions (elu, selu, hardswish, mish, softplus). We also provide a fused NEON Adam optimizer step that performs the momentum, velocity, and weight updates in a single pass over memory.

Parallelism. Operations on tensors larger than a configurable threshold (default: 2M elements) are parallelized via `rayon`. Multi-head attention parallelizes across heads.

3.4 Metal GPU Backend

The optional Metal backend (`-features metal`) provides GPU acceleration via 108 Metal Shading Language (MSL) compute kernels embedded as compile-time string constants.

Architecture. A singleton `GpuContext` manages the Metal device, command queue, 108 pre-compiled pipeline states, and a buffer pool. It exposes 41 typed dispatch methods (e.g., `dispatch_matmul`, `dispatch_attention_causal`, `dispatch_sparse_matmul_24`).

Kernel categories. The 108 kernels cover:

- Element-wise binary and unary operations (forward and backward)
- Activation functions (GELU, SiLU, ELU, etc.) with backward kernels
- Reductions (sum, mean, max, min, var, argmax per axis)
- Fused operations (matmul + bias + GELU, add + layer norm, add + GELU)
- Attention-specific kernels (QKV reshape for GQA, RoPE2D, output reshape)
- Causal masked scaled dot-product attention with grouped-query support
- Sparse matrix multiplication (2:4 structured sparsity)
- Quantized matrix multiplication (int8 dequantize-and-multiply)

Operator fusion. Fused kernels eliminate intermediate tensor allocations. For example, `matmul_bias_gelu` computes $\text{GELU}(xW+b)$ in a single kernel launch, avoiding two intermediate buffers. Similarly, `add_layer_norm` fuses residual addition with layer normalization.

Heterogeneous scheduling. For models like MUSt3R that process two views, Peregrine can dispatch one view's computation to the GPU while processing the other on the CPU, synchronized via `MTLSharedEvent`. This heterogeneous pipeline achieves up to 36% speedup over CPU-only execution at 512×384 resolution.

Thermal-aware scheduling. On macOS, Peregrine monitors thermal state via Darwin notifications. Under thermal pressure, GPU dispatch can be deferred to avoid throttling.

GELU precision fix. We discovered that Metal's fast-math `tanh()` produces NaN for certain input magnitudes (approximately $|x| > 10.5$). The GELU kernel uses `precise::tanh()` instead, ensuring bit-identical CPU–GPU output.

3.5 Quantization and Sparsity

Int8 quantization. Peregrine supports per-column symmetric weight quantization and per-row dynamic activation quantization. CPU inference uses a NEON int8 GEMM kernel; the Metal backend provides a fused dequantize-and-multiply kernel.

2:4 structured sparsity. Weights are pruned to a 2:4 pattern (2 non-zeros per group of 4 elements). Indices are nibble-packed (4 bits per element) for memory efficiency. CPU inference uses a NEON sparse GEMM with scalar 16×16 tiles; the Metal backend uses simdgroup 32×32 tiles. This yields a $1.78\times$ bandwidth reduction.

3.6 Model Format Support

Peregrine includes parsers for three model weight formats, all implemented in pure Rust without external parsing libraries:

- **Peregrine binary format:** A compact format storing tensor names, shapes, and f32/int8/sparse data. Used for converted weights.
- **GGUF:** The format used by llama.cpp, supporting Q8_0, Q4_0, Q4_1, F16, and F32 quantization types with on-the-fly dequantization.
- **SafeTensors:** The Hugging Face format, with mmap-based loading on Unix, F32/F16/BF16 dequantization, and a handwritten JSON header parser.

Additionally, Peregrine includes tokenizer parsers for SentencePiece (protobuf format, pure Rust) and Hugging Face `tokenizer.json` (byte-level BPE, pure Rust).

4 Supported Operations

Table 1 summarizes the operation categories. All operations support reverse-mode autograd where mathematically applicable.

Table 1: Operation categories in Peregrine. All support autograd and broadcasting where applicable.

Category	Operations
Arithmetic (10)	add, sub, mul, div, neg, scale, maximum, minimum, power, logaddexp
Math (32)	exp, log, sqrt, abs, sin, cos, tanh, sinh, cosh, reciprocal, square, rsqrt, erf, erfinv, expm1, log2, log10, log1p, arcsin, arccos, arctan, arcsinh, arccosh, arctanh, arctan2, degrees, radians, floor, ceil, round, sign, pow
Activations (21)	relu, sigmoid, gelu, silu, softplus, mish, leaky_relu, elu, hard_tanh, relu6, hardswish, softsign, log_sigmoid, selu, celu, gelu_fast, softmin, glu, hard_shrink, soft_shrink, PReLU
Reductions (21)	sum, mean, softmax, log_softmax, sum/mean/max/min_axis, var, std, prod_axis, logsumexp, cumsum, cumprod, argmax/argmin_axis, topk, sort, argsort, any, all
Shape (22)	reshape, transpose, squeeze, unsqueeze, concat, select, flatten, stack, split, tril, triu, repeat, tile, pad, roll, take, diagonal, diag, trace, outer, inner, broadcast_to
Layers (23)	matmul, matmul_quantized, matmul_sparse_24, conv1d, conv2d, conv2d_strided, conv_transpose1d/2d, max/avg_pool2d, add_bias, batch/layer/rms/group/instance_norm, upsample, index_select, fused matmul+bias+gelu, fused add+layernorm
Loss (14)	cross_entropy, mse, l1, nll, smooth_l1, huber, kl_div, bce_with_logits, cosine_similarity, triplet, hinge, log_cosh, margin_ranking, gaussian_nll

5 Neural Network Modules

Peregrine provides a `Module` trait with `forward()`, `params()`, and `named_params()` methods, along with train/eval mode toggling. Table 2 lists the implemented modules.

Table 2: Neural network modules in Peregrine.

Category	Modules
Linear	Linear, Embedding
Attention	MultiHeadAttention (with GQA, causal masking, RoPE)
Recurrent	RNN, LSTM, GRU
Convolution	Conv1d, Conv2d, ConvTranspose1d, ConvTranspose2d
Pooling	MaxPool1d/2d, AvgPool1d/2d, AdaptiveAvgPool
Normalization	BatchNorm1d/2d, LayerNorm, GroupNorm, InstanceNorm, RMSNorm
Regularization	Dropout, Dropout2d, AlphaDropout
Padding	Constant, Reflect, Replicate
Other	Upsample (nearest/bilinear), PixelShuffle, Flatten, Sequential

Optimizers. Eight optimizers are implemented: SGD (with momentum, Nesterov, weight decay), Adam/AdamW, RMSprop, Adagrad, Adamax, AdaDelta, Lion [12], and Adafactor [13]. Learning rate schedulers include step decay, cosine annealing, warmup linear, exponential decay, and composable schedules via `Join`.

Attention. The core attention module implements grouped-query attention (GQA) [10] with a `StandardKVCache` supporting append and rollback operations. Attention masks include None, Causal, CausalSlidingWindow,

and LocalGlobal. Post-score transforms include LogitCap ($c \cdot \tanh(s/c)$, used by Grok-1 with $c = 30$). The CPU path uses NEON dot products for QK^T, Accelerate `vvexpf` for softmax, and NEON weighted accumulation for the value projection. The Metal backend provides a causal masked SDPA kernel with GQA grouping, plus dedicated QKV reshape and RoPE2D kernels.

Speculative decoding. Peregrine implements the Leviathan et al. [11] speculative decoding algorithm: a draft model proposes N tokens, the target model verifies them in a single batched forward pass, and tokens are accepted with probability $\min(1, q(x)/p(x))$. Rejected tokens trigger resampling from $\max(0, q - p)$ and KV cache rollback.

6 Example Models

Peregrine includes complete implementations of eight models spanning vision, language, reinforcement learning, and 3D reconstruction. Table 3 provides an overview.

Table 3: Models implemented in Peregrine. All run inference; MNIST, RT-DETR, RL, and MOBA also train.

Model	Domain	Params	Key features
Grok-1	Language (MoE)	314B	GQA, 8 experts top-2, SwiGLU, logit cap
DeepSeek-V3/R1	Language (MoE)	671B	MLA, 256+1 experts top-8, YaRN RoPE
Llama 3.2	Language	varies	GGUF/SafeTensors/HF Hub, streaming
MUSt3R	3D reconstruction	423M	ViT-L/B, server mode, Metal GPU
RT-DETR	Detection	1.2M	ResNet + transformer, COCO, wandb
MNIST	Classification	109K	MLP, 97.5% test accuracy
RL Demo	RL	varies	PPO, DQN, REINFORCE
MOBA	Game AI	varies	3v3 LSTM PPO, self-play

6.1 Large Language Models

Grok-1 (314B parameters). A Mixture-of-Experts transformer with 64 layers, 6144 embedding dimension, grouped-query attention (48 query heads, 8 key-value heads, 128 head dimension), 8 experts with top-2 routing, SwiGLU FFN (32768 intermediate), RoPE ($\theta = 500{,}000$), RMSNorm, and logit capping ($30 \cdot \tanh(s/30)$). Includes a pure-Rust SentencePiece tokenizer (131,072 vocabulary, protobuf parser, BPE encode/decode matching Python exactly).

DeepSeek-V3/R1 (671B parameters). A Mixture-of-Experts transformer with 61 layers (3 dense + 58 MoE), 7168 embedding dimension, 128 attention heads, 256 routed experts plus 1 shared expert, top-8 activation with group-limited selection (8 groups). Features Multi-head Latent Attention (MLA) [14] with compressed KV cache via LoRA (`q_lora_rank`=1536, `kv_lora_rank`=512), caching only 576 floats per token per layer versus 12,800 in standard multi-head attention—a 22× compression. Uses YaRN RoPE [15] for context extension and sigmoid routing.

Llama 3.2. Supports GGUF, SafeTensors, and Hugging Face Hub download. Features streaming decode with per-token latency tracking, sustained profiling (p50/p95/p99 latencies grouped by thermal state), chunked prefill for large prompts, multi-request scheduling, and top-p (nucleus) sampling.

6.2 Vision and 3D Models

MUSt3R (423M parameters). A stereo 3D reconstruction model [16] with a ViT-Large encoder (1024 dimensions, 24 blocks, 16 heads) and a ViT-Base decoder (768 dimensions, 12 blocks, 12 heads). Supports variable resolution (224×224, 512×384), server mode (weight loading amortized across pairs via stdin/stdout),

and parallel multi-pair processing. The Metal GPU backend dispatches encoder attention to the GPU with heterogeneous CPU–GPU scheduling via `MTLSharedEvent`.

RT-DETR. A real-time detection transformer with a ResNet backbone, multi-scale features ($128^2, 64^2, 32^2$), transformer encoder/decoder, Hungarian matching, and COCO dataset loading with Weights & Biases integration.

6.3 Reinforcement Learning

Peregrine includes PPO, DQN, and REINFORCE implementations with 10 built-in environments (CartPole, MountainCar, GridWorld, FrozenLake, and six reasoning tasks), as well as a MOBA 3v3 game with LSTM-based recurrent actor-critic and self-play training on a 32×16 grid with heroes, towers, creeps, and bases. Both produce HTML replay animations; the MOBA also exports MP4 video.

7 Benchmarks

All benchmarks were run on an Apple M1 Max (10 cores, 64 GB RAM) running macOS. We compare against PyTorch 2.10.0, MLX 0.30.6, TensorFlow 2.20.0, JAX 0.9.0.1, and tinygrad. All measurements are CPU-only, reporting the median of 20–50 iterations after warmup.

7.1 Operator-Level Benchmarks

We benchmark 141 operations covering matrix multiplication, element-wise ops, activations, reductions, loss functions, linear algebra, FFT, optimizers, convolutions, normalization, recurrent networks, and random number generation. Full results are available in the supplementary material; representative results are in Table 4.

Summary statistics. Peregrine wins 128 of 141 operations (90.8%). The geometric mean ratio versus PyTorch is 0.44× (Peregrine is 2.3× faster on average). Against other frameworks: MLX 0.32×, TensorFlow 0.24×, JAX 0.29×, tinygrad 0.04×.

Where PyTorch wins. PyTorch is faster on LSTM (810 μs vs. 930 μs, 1.15×) and fused matmul+bias+GELU at large sizes (849 μs vs. 986 μs, 1.16×). The LSTM gap likely reflects PyTorch's highly optimized cuDNN-derived CPU kernel.

Sources of speedup. The large speedups on element-wise operations (3–5× on add, relu, etc.) and reductions (8–32× on softmax, cross-entropy) primarily reflect the elimination of Python/C++ dispatch overhead. Peregrine's Rust code calls NEON intrinsics or Accelerate functions directly, bypassing the Python interpreter, ATen dispatcher, and intermediate tensor allocation. For matmul, all frameworks route to the same Apple Accelerate BLAS, and performance converges; Peregrine's advantage at smaller sizes (128×128: 1.3×) reflects lower dispatch overhead.

7.2 End-to-End Inference: MUSt3R

Table 5 shows MUSt3R inference times for a single image pair.

At 224×224, Peregrine CPU is 12% faster than PyTorch; the GPU+Pipeline mode is 19% faster. At 512×384, the GPU+Pipeline achieves 36% lower latency than PyTorch. Weight loading is 2.7× faster (0.6s vs. 1.6s).

The GPU+Pipeline mode uses heterogeneous scheduling: the encoder for one view runs on the Metal GPU while the other view runs on the CPU, synchronized via `MTLSharedEvent`. At 512×384, this reduces encoder time from 1341 ms (CPU) to 521 ms (pipeline), as the GPU encoder (53 ms) runs concurrently with the CPU encoder.

Table 4: Representative operator benchmarks (μs, lower is better). Best in **bold**. Full 141-op results at `https://github.com/etam103/peregrine`.

Operation	Peregrine	PyTorch	MLX	TF	tinygrad	JAX
matmul 128 × 128	**4.7**	6.2	24.0	95.7	418.9	80.4
matmul 256 × 256	**29.5**	31.8	47.8	197.7	423.8	171.3
matmul 512 × 512	**134.8**	140.0	172.1	682.6	433.2	518.9
add 100k	**12.5**	41.1	32.3	50.1	190.5	33.4
mul 100k	**12.5**	39.0	32.0	40.7	192.3	28.4
relu 100k	**8.5**	38.5	30.5	39.7	338.9	97.7
gelu 100k	**56.3**	74.0	146.2	249.2	844.8	219.3
silu 100k	**47.0**	68.8	89.8	226.0	370.4	78.6
mish 100k	**136.7**	307.9	387.4	244.6	—	258.7
hardswish 100k	**10.0**	40.7	65.0	220.3	—	37.4
softmax 8 × 128	**1.1**	35.2	18.4	11.3	609.4	30.9
var 256 × 512	**45.7**	271.5	—	—	—	—
cross_entropy 64 × 10	**2.5**	40.1	24.8	614.7	3622.5	56.7
huber loss 64 × 10	**0.3**	4.7	34.8	238.2	—	48.8
cosine sim 64 × 64	**1.8**	10.3	104.3	236.0	—	55.8
layernorm 196 × 768	**72.0**	101.0	—	1224.3	1137.2	259.0
rmsnorm 64 × 512	**18.7**	63.5	32.0	439.4	—	78.2
groupnorm $4{\times}64{\times}16^2$	**21.4**	52.1	231.0	754.5	—	271.6
cholesky 128 × 128	**14.2**	46.4	26.7	58.5	—	37.7
inv 128 × 128	**48.5**	62.0	84.6	139.1	—	89.5
solve 128 × 128	**36.2**	44.8	187.2	76.7	—	85.8
rfft 1k	**2.2**	4.4	19.0	42.7	—	60.5
rand_uniform 100k	**60.2**	257.6	522.1	126.2	—	593.4
train step (batch 64)	**747**	1255	810	8444	24421	5209
adam step	**744**	1269	—	—	—	—

Table 5: MUSt3R single-pair inference time (seconds, lower is better).

Resolution	Peregrine CPU	Peregrine GPU+Pipe	PyTorch CPU
224 × 224	0.59	**0.54**	0.67
512 × 384	1.95	**1.44**	2.26

7.3 Optimization Techniques

The following techniques contribute to Peregrine's performance:

1. **NEON 8-wide unrolled kernels**: Two 128-bit registers per iteration, eliminating branch overhead for the common case.
2. **Apple Accelerate vForce**: Vectorized transcendental functions avoid the overhead of scalar libm calls.
3. **Zero-copy strided SGEMM**: Attention computes QK^T and softmax$(S)V$ via offset-based BLAS calls without reshaping tensors.
4. **Fused Metal kernels**: matmul+bias+GELU and add+layernorm eliminate intermediate GPU buffers.
5. **Buffer pooling**: Both CPU and GPU backends reuse allocated buffers to reduce allocation overhead.
6. **Lazy GPU synchronization**: Metal barriers are skipped when the consuming operation does not require the gradient of its input.
7. **Fused NEON Adam**: The optimizer step updates momentum, velocity, and weights in a single pass, improving cache utilization.

8 Testing and Correctness

Peregrine includes 633 tests organized into the following categories:

- **PyTorch parity tests (23):** Cross-validate matmul, softmax, layer normalization, cross-entropy loss, 14 element-wise operations, Adam optimizer, and a full 10-step MLP training loop against PyTorch reference data. All pass within 10^{-4} to 10^{-7} absolute error.
- **Activation tests (34):** Each of 34 activation functions validated against PyTorch reference outputs.
- **Attention tests (8):** Multi-head attention, GQA, MQA, causal masking, sliding window, logit capping, cache rollback.
- **Metal parity tests (35):** CPU vs. GPU output equality for all dispatched operations, including heterogeneous execution, causal SDPA, GQA, and 2:4 sparse matmul.
- **Speculative decoding tests (4):** Greedy acceptance, stochastic acceptance, EOS handling, cache rollback.
- **Quantization and sparsity tests (8):** Roundtrip accuracy, CPU–GPU parity, serialization.
- **Format parser tests (5):** GGUF Q8_0/Q4_0/Q4_1 dequantization, invalid input handling.
- **Unit tests (352+):** Individual tensor operations, shape manipulation, broadcasting, edge cases.

Reference data is generated by a Python script (`tests/generate_reference.py`) that computes the same operations in PyTorch and saves the results as binary tensor files.

9 Design Decisions and Lessons Learned

Rust for ML. Rust's ownership model eliminates data races in parallel code (e.g., rayon-based multi-head attention) without runtime overhead. The lack of a garbage collector provides predictable latency—per-token decode in Llama has a p99/p50 ratio below 1.2 under nominal thermal conditions. However, the autograd graph requires interior mutability (`Rc<RefCell<...>>`), which is single-threaded by design. An `Arc<Mutex>` alternative would add lock overhead on every tensor access.

Apple AMX investigation. We reverse-engineered Apple's undocumented AMX coprocessor instructions and implemented a working SGEMM kernel. Key findings: SET/CLR use opcode 17 with a 3-NOP sled (not opcodes 0/1 as previously documented); load opcodes are LDX=0, LDY=1, LDZ=4, STZ=5 (not 2, 3, 7, 6); the Z accumulator uses stride-4 rows for f32. Our naive kernel achieved 8.5 GFLOPS versus approximately 1,000 GFLOPS for `cblas_sgemm`—a 70–130× gap due to Apple's proprietary tiling. The conclusion: AMX is accessible from userspace on macOS, but `cblas_sgemm` is the only practical way to use it. The M4 chip's ARM SME (Scalable Matrix Extension) may offer a documented alternative if Rust adds stable SME intrinsics.

Metal fast-math. Metal's fast-math compilation mode introduces NaN for certain `tanh()` inputs (approximately $|x| > 10.5$). Since GELU uses $\tanh(\sqrt{2/\pi} \cdot (x + 0.044715x^3))$, NaN from tanh propagates through the entire layer. The failure was intermittent and initially misdiagnosed as a weight conversion bug. The fix: replace `tanh()` with `precise::tanh()` in both forward and backward GELU kernels. After this change, CPU and GPU produce byte-identical output. This bug is not documented in Apple's Metal Shading Language specification.

Neural Engine dead end. We investigated Apple's Neural Engine (ANE) as a third compute backend via reverse-engineered private APIs (`_ANEClient`, `_ANECompiler`). The ANE achieved only 11.2% utilization (1.78 of 15.8 TFLOPS) on M4, with additional limitations: weights baked at compile time, ∼119 compile limit per process, ∼8–10 MB SRAM, fp16 only, and no causal SDPA. Combined with reliance on private APIs that break across macOS versions, we concluded the ANE is not viable as a general compute backend.

Weight layout conventions. PyTorch stores linear layer weights as $[\text{out}, \text{in}]$; Peregrine uses $[\text{in}, \text{out}]$. Weight conversion scripts must transpose 2D linear weights, which was a source of subtle bugs in early MUSt3R development.

Dead-end optimizations. We tried and rejected: (1) NEON `vrecpeq/vrecpsq` for fast reciprocal—slower than `vdivq_f32` on Apple Silicon's fast divider; (2) NEON polynomial sigmoid—doesn't converge on $[-8, 8]$ for f32; (3) NEON `arccos`—slower than Apple's `vvacosf`; (4) Accelerate `vvexpf` for SiLU—slower than a Cephes polynomial fast-exp.

10 Limitations

- **Platform:** Peregrine currently targets only macOS on Apple Silicon. Porting to Linux would require replacing Accelerate (with OpenBLAS or MKL), Metal (with Vulkan or CUDA), and Darwin notifications. The core tensor and autograd code is platform-independent.
- **Precision:** All computation uses f32. Half-precision (f16/bf16) is supported only for weight loading and dequantization, not for compute kernels.
- **Training at scale:** While Peregrine supports training (autograd, optimizers, RL), it has not been validated on large-scale distributed training workloads.
- **Attention autograd:** The attention forward pass breaks the autograd graph for efficiency; only the output projection trains via backpropagation.
- **GPU training overhead:** GPU training is slower than CPU at small batch sizes (e.g., 64) due to synchronous reduction in `mean()`. Larger batch sizes favor GPU.
- **No REPL:** ML development is exploratory. Python bindings via PyO3 partially address this but add a layer Peregrine was designed to eliminate.

11 Conclusion

We presented Peregrine, a deep learning framework built from scratch in approximately 37,900 lines of Rust. By eliminating framework overhead and leveraging Apple Silicon-specific optimizations—Accelerate BLAS, hand-tuned NEON SIMD, and 108 Metal GPU compute shaders—Peregrine achieves a 2.3× geometric mean speedup over PyTorch across 141 operator benchmarks and 12–36% faster end-to-end inference on the 423M-parameter MUSt3R model.

The framework supports models up to 671B parameters, passes 633 tests ensuring numerical correctness against PyTorch, and includes production features such as speculative decoding, thermal-aware scheduling, heterogeneous CPU–GPU pipelines, and request batching. Despite this breadth, the entire codebase is small enough to read in its entirety—every operation, from the NEON add kernel to the Metal SDPA shader to the SentencePiece tokenizer, is implemented in one place with no hidden dependencies.

Peregrine is open-source under the MIT license at `https://github.com/etam103/peregrine`.

References

[1] A. Paszke, S. Gross, F. Massa, et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *NeurIPS*, 2019.

[2] M. Abadi, P. Barham, J. Chen, et al. TensorFlow: A System for Large-Scale Machine Learning. In *OSDI*, 2016.

[3] J. Bradbury, R. Frostig, P. Hawkins, et al. JAX: Composable Transformations of Python+NumPy Programs. 2018. `http://github.com/google/jax`

[4] A. Hannun, et al. MLX: An Array Framework for Apple Silicon. 2023. https://github.com/ml-explore/mlx

[5] G. Hotz. tinygrad. https://github.com/tinygrad/tinygrad

[6] Hugging Face. Candle: Minimalist ML framework for Rust. https://github.com/huggingface/candle

[7] Tracel AI. Burn: A comprehensive deep learning framework in Rust. https://github.com/tracel-ai/burn

[8] G. Gerganov. llama.cpp. https://github.com/ggerganov/llama.cpp

[9] Microsoft. ONNX Runtime. https://github.com/microsoft/onnxruntime

[10] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *EMNLP*, 2023.

[11] Y. Leviathan, M. Kalman, and Y. Matias. Fast Inference from Transformers via Speculative Decoding. In *ICML*, 2023.

[12] X. Chen, C. Liang, D. Huang, et al. Symbolic Discovery of Optimization Algorithms. In *NeurIPS*, 2023.

[13] N. Shazeer and M. Stern. Adafactor: Adaptive Learning Rates with Sublinear Memory Cost. In *ICML*, 2018.

[14] DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. *arXiv:2405.04434*, 2024.

[15] B. Peng, J. Quesnelle, H. Fan, and E. Shippole. YaRN: Efficient Context Window Extension of Large Language Models. *arXiv:2309.00071*, 2023.

[16] Y. Wang, et al. MUSt3R: Multi-View Stereo 3D Reconstruction. 2024.